Near-Maximum Circuit Lower Bounds from Range Avoidance

A one-lecture proof via Li's postorder idea, with conceptual and historical context

August 2026

Abstract

We give a lecture-oriented proof of an almost-maximal Boolean circuit lower bound in the second level of exponential time. The proof is extracted from Zeyong Li's 2023–24 simplification of the Chen–Hirahara–Ren breakthrough, but is deliberately specialized to the easier target $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$. This lets us omit the symmetric-class selector and all iterative win–win machinery. The central idea is strikingly elementary: apply a circuit down a binary GGM tree, invert the tree in *postorder*, and observe that the enormous canonical computation history is a union of only linearly many fully generated subtrees. Hence the history has a short description and can be universally verified. Combining this canonical Range-Avoidance procedure with the truth-table generator for small circuits gives a language in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$ requiring more than $2^n/n$ gates at every sufficiently large length. We also explain the route from Shannon and Kannan through Range Avoidance, the stronger $\mathsf{S}_2\mathsf{E}$ theorem of Li, the Linear Ordering Principle perspective of Korten–Pitassi, and the current frontier.

The conceptual punchline. Shannon's counting proof says that hard truth tables exist. The modern viewpoint asks for a *canonical missing output* of the circuit that enumerates all easy truth tables. Li's postorder trick makes one such missing output describable by an $\exists\forall$ certificate of polynomial length. After scaling the construction from truth-table length $N = 2^n$ back to input length n, polynomial time in N becomes exponential time in n, giving a near-maximum lower bound in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$.

1 The theorem and the proof at a glance

Throughout, log means $\log_2$. Circuit size is the number of non-input gates, over the full binary Boolean basis. Let $C_L(n)$ denote the minimum size of a circuit computing the characteristic function of a language L on n-bit inputs.

Theorem 1.1 (Lecture form of the Li lower bound)**.** *There is a language $L \in \Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$ such that, for every sufficiently large n,*

$$C_L(n) > \frac{2^n}{n}.$$

Consequently L has $(1 - o(1))$ times the maximum possible circuit complexity at every sufficiently large input length.

The second sentence uses the classical Shannon–Lupanov estimate

$$\max_{f:\{0,1\}^n \to \{0,1\}} C(f) = (1 + o(1))\frac{2^n}{n},$$

and in fact the sharper bounds of Frandsen–Miltersen [8].

The proof has four ingredients.

1. **Truth-table generator.** Descriptions of all size-s circuits, for $s = 2^n/n$, fit into fewer than 2^n bits. Thus there is an expanding circuit

$$G_n : \{0,1\}^{2^n-1} \longrightarrow \{0,1\}^{2^n}$$

   whose range contains the truth table of every n-input circuit of size at most $2^n/n$.

2. **Range Avoidance.** If we canonically find

$$y_n \notin \mathrm{Im}(G_n),$$

   then the Boolean function with truth table y_n requires more than $2^n/n$ gates.

3. **Canonical $\exists\forall$ Range Avoidance.** For every circuit

$$G : \{0,1\}^m \to \{0,1\}^{m+1},$$

   there is a *unique* polynomial-length certificate π_G, universally checkable in polynomial time, from which one reads a canonical $y_G \notin \mathrm{Im}(G)$.

4. **Scale back.** For $m = 2^n - 1$, "polynomial in m" is $2^{O(n)}$. Thus the unique certificate for G_n gives both a $\Sigma_2\mathsf{E}$ and a $\Pi_2\mathsf{E}$ definition of the language whose length-n truth table is y_n.

The only genuinely new-looking step is item 3. Sections 4–9 prove it from scratch.

2 Why this problem is natural: a short history

1949–1999: existence versus uniform construction. Shannon's counting argument showed that almost all Boolean functions need exponentially large circuits [1]. Lupanov sharpened this to the asymptotically tight scale $2^n/n$ [2]. Kannan then asked how low in the uniform complexity hierarchy one can place functions with large circuit complexity: his 1982 work put a maximum-complexity function at the third level of exponential time, and proved the famous fixed-polynomial lower bounds in the polynomial hierarchy [3]. Miltersen, Vinodchandran, and Watanabe observed that binary search lowers the exact-maximum construction to $\Delta_3\mathsf{E} = \mathsf{E}^{\Sigma_2\mathsf{P}}$ [6]. For more than twenty years, no class substantially below this was known to contain a function of exponential circuit complexity.

The issue is not existence. Given unlimited resources, one can enumerate truth tables, test their circuit complexity, and choose the first hardest one. The issue is *uniform canonicalization*: how little computation suffices to specify, at every length, one particular hard truth table?

This question became more important after hardness-versus-randomness: strong circuit lower bounds for E imply powerful derandomization consequences [4, 5]. Thus the uniform complexity of hard functions became a proxy for one of the central obstacles in derandomization.

2004–2021: the Range-Avoidance reformulation. Jeřábek's work on the dual weak pigeonhole principle connected the impossibility of a surjection from a smaller domain onto a larger range with derandomization and bounded arithmetic [7]. Korten reinterpreted this viewpoint algorithmically: the search problem Avoid, which asks for a point outside the range of an expanding circuit, is a universal explicit-construction problem, and hard truth-table construction is complete for the corresponding explicit-construction class under P^{NP} reductions [10].

The key reduction is almost tautological. Let $\mathsf{TT}_{n,s}$ map a short description of an n-input size-s circuit to its 2^n-bit truth table. Then

$$y \notin \mathrm{Im}(\mathsf{TT}_{n,s}) \quad \Longrightarrow \quad C(y) > s.$$

So "construct a hard function" becomes "find a missing output."

2023–24: the breakthrough. Chen, Hirahara, and Ren gave a single-valued algorithm for Range Avoidance in symmetric polynomial time at infinitely many lengths; this yielded a near-maximum lower bound for $\mathsf{S_2E}/_1$ [11]. Their proof used an iterative win–win to locate a scale at which a huge inversion history had a short description. Li then noticed that a simple change of traversal order—use *postorder*—makes the worst-case history itself succinct. This removes the iterative win–win and the advice bit, and yields an almost-everywhere $2^n/n$-type lower bound already in $\mathsf{S_2E}$, hence also in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$ [12].

For a one-lecture proof, $\mathsf{S_2E}$ is slightly more than we need. Li's first, easier result is a canonical $\exists\forall$ algorithm for the relevant Range-Avoidance instance. That already gives Theorem 1.1. The additional $\mathsf{S_2E}$ selector is conceptually nice but unnecessary for the lower bound in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$.

Korten and Pitassi subsequently gave a structural explanation of Li's canonicalization via the *Linear Ordering Principle* (LOP): tree iteration first prepares a special family of candidate missing outputs, and a succinct total order then singles out a canonical one [13]. We return to this perspective in Section 12.

3 The Shannon scale and a compact encoding of circuits

The exact constant 1 in $2^n/n$ is not automatic. A naive gate-by-gate circuit description uses about $2\log s$ bits just to name two predecessors, which loses a constant factor. Frandsen and Miltersen observed a compact *stack-program* representation that has essentially optimal description length [8].

Lemma 3.1 (Stack-program encoding)**.** *Every n-input Boolean circuit with s non-input gates can be represented by a stack program using exactly s binary-operation instructions and $s+1$ push instructions. The whole program can be encoded using at most*

$$L_{n,s} := (s+1)\big(7 + \lceil \log(n+s) \rceil\big)$$

bits.

Proof. Traverse the circuit recursively from the output. Whenever a gate is needed for the first time, recursively arrange to push its two predecessors and then execute its binary Boolean operation. If a

previously computed gate is needed again, simply push a reference to its earlier result. Thus each circuit gate gives one binary-operation instruction.

A push increases stack height by one; a binary operation consumes two stack items and replaces them by one, decreasing height by one. Starting from height zero and ending with the single output value, a program with s binary operations must therefore have exactly $s + 1$ pushes.

A push argument names one of the n inputs or one of the at most s previous computed values, costing $\lceil \log(n + s) \rceil$ bits. A constant number of additional bits identifies "push" versus a binary Boolean operation and, in the latter case, one of the sixteen binary operations. Seven bits of overhead per push/operation pair suffice. This gives the stated bound. The construction is the one used by Frandsen–Miltersen. □

Corollary 3.2 (Truth-table generator at the Shannon scale)**.** *Let $N = 2^n$ and $s = \lfloor N/n \rfloor$. For all sufficiently large n, $L_{n,s} < N$. Moreover, in time* poly(N) *one can construct a Boolean circuit*

$$\mathsf{TT}_{n,s} : \{0,1\}^{L_{n,s}} \to \{0,1\}^N$$

whose range contains the truth table of every n-input circuit of size at most s.

Proof. The map $\mathsf{TT}_{n,s}$ simply simulates the encoded stack program on each of the N possible inputs in parallel. Its size and uniform construction time are polynomial in N.

For the length estimate, $s \le N/n$ and, for large n,

$$\log(n + s) \le n - \log n + O(1).$$

Therefore

$$L_{n,s} \le \left(\frac{N}{n} + 1 \right) (n - \log n + O(1)) < N$$

for all sufficiently large n. □

Padding unused input bits, we may regard $\mathsf{TT}_{n,s}$ as a circuit

$$G_n : \{0,1\}^{N-1} \to \{0,1\}^N.$$

Any missing output of G_n is therefore a truth table requiring more than N/n gates.

The modern reframing. The hard-function problem has become a total search problem. The circuit G_n has fewer inputs than outputs, so the pigeonhole principle guarantees a missing output. The lower-bound problem is no longer to *certify* that a guessed truth table is hard. It is to *canonically find* one point outside an efficiently described range.

4 First simplification: from stretch one to stretch two

Li's clean postorder argument is most naturally stated for a circuit $C : \{0,1\}^m \to \{0,1\}^{2m}$. We first reduce an arbitrary weak-stretch instance $G : \{0,1\}^m \to \{0,1\}^{m+1}$ to that case. The following elementary gadget is a convenient explicit version of the short-stretch amplification underlying Korten's reduction.

Lemma 4.1 (Stretch amplification)**.** *Given a circuit $G : \{0,1\}^m \to \{0,1\}^{m+1}$, one can construct in polynomial time a circuit*

$$D : \{0,1\}^m \to \{0,1\}^{2m}$$

of size $O(m\,|G|)$ with the following property. From any $y \notin \mathrm{Im}(D)$, one can canonically obtain a string $z \notin \mathrm{Im}(G)$ by an m-step reverse computation. The reverse computation has a unique polynomial-length $\exists\forall$ certificate.

Proof. Write

$$G(x) = \mathrm{next}(x)\ \mathrm{bit}(x),$$

where $\mathrm{next}(x) \in \{0,1\}^m$ and $\mathrm{bit}(x) \in \{0,1\}$. Starting from $x_0 = x$, iterate

$$G(x_{i-1}) = x_i b_i, \qquad i = 1, \ldots, m,$$

and define

$$D(x_0) := x_m\, b_m b_{m-1} \cdots b_1.$$

This has exactly $2m$ output bits and is computed by m serial copies of G.

Now let

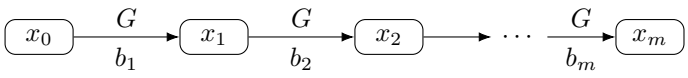
$$y = x_m\, b_m b_{m-1} \cdots b_1 \notin \mathrm{Im}(D).$$

Run backward. At stage $i = m, m-1, \ldots, 1$, look for a preimage x_{i-1} satisfying

$$G(x_{i-1}) = x_i b_i.$$

If none exists, then $x_i b_i \notin \mathrm{Im}(G)$, and we stop. If a preimage exists, choose the lexicographically first one and continue. We must eventually stop: otherwise we recover x_0 with $D(x_0) = y$, contradicting $y \notin \mathrm{Im}(D)$.

The transcript is canonical because every successful reverse step chooses the lexicographically first preimage. It is universally checkable: for a claimed preimage u, check $G(u) = x_i b_i$ and, universally over every $u' < u$, check $G(u') \neq x_i b_i$; at the failure stage, universally check that no u maps to the target. A universal challenge can name the stage and the candidate u, so the whole transcript has one existential block followed by one universal block. □



$D(x_0) = x_m\, b_m \cdots b_1$; a missing output of D is reversed until the first missing output of G appears.

This reduction is cheap enough that we may henceforth focus on the doubling case.

5 The GGM tree: turn one missing point into a forced failure

Fix a circuit

$$C : \{0,1\}^m \to \{0,1\}^{2m}.$$

Write $C(x) = C_L(x)C_R(x)$, with each half m bits. The GGM tree iteration [9] places a seed x at the root of a complete binary tree and, at every internal node carrying v, gives its two children the values $C_L(v)$ and $C_R(v)$.

Take the tree height to be

$$h = 2m + 1.$$

There are 2^h leaves, each holding m bits. Pair the leaves into consecutive siblings. Now define the explicit leaf string

$$F_m := 00\cdots00\ 00\cdots01\ \cdots\ 11\cdots11,$$

the concatenation, in lexicographic order, of *all* $2m$-bit strings. Its length is

$$2m \cdot 2^{2m} = m \cdot 2^{2m+1},$$

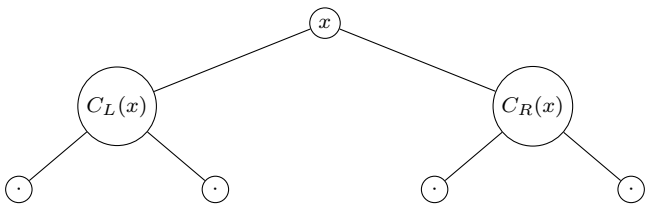
exactly the number of leaf bits in the height-$2m+1$ tree.

Lemma 5.1 (The explicit leaf string is impossible)**.** *No root seed produces the leaf string F_m under the GGM iteration of C.*

Proof. If some seed produced F_m, then every consecutive sibling pair at the bottom level would equal $C(v)$ for its parent v, and hence would lie in $\mathrm{Im}(C)$. But the sibling pairs in F_m enumerate every string in $\{0,1\}^{2m}$, whereas

$$|\,\mathrm{Im}(C)| \le 2^m < 2^{2m}.$$

At least one pair is missing from the range of C, a contradiction. □



Iterate C down to height $2m+1$. At the bottom, every sibling pair must be an output of C; F_m deliberately lists all $2m$-bit strings.

6 Invert the tree backward—but in postorder

Assign the leaves according to F_m. We now try to reconstruct internal labels bottom-up. The procedure is deterministic once we insist on lexicographically first preimages.

Reverse-GGM procedure. Traverse the internal vertices in postorder: left subtree, then right subtree, then the vertex itself. At a vertex u whose children already carry $a, b \in \{0,1\}^m$, search for an $x \in \{0,1\}^m$ with

$$C(x) = ab.$$

If one exists, write the lexicographically first such x at u. If none exists, stop and output ab.

The procedure must stop before the root: if every internal node could be assigned, the root label would be a seed generating F_m, contradicting Theorem 5.1. Thus it outputs a string outside $\mathrm{Im}(C)$.

Executed literally, this uses an NP oracle to find each lexicographically first preimage and takes exponential time, because the tree is enormous. The breakthrough is not that the procedure exists. It is that its entire canonical history has a tiny description.

Why traversal order matters. A generic bottom-up traversal can leave a wildly fragmented set of already processed vertices. Postorder has a rigid geometry: when the algorithm first fails at a vertex u^*, the processed portion of the tree is a union of only $O(h)$ complete subtrees. A complete processed subtree is fully determined by its root label, because all descendants are generated forward by C.

7 Li's postorder lemma: the huge history has a short description

Let u^* be the first internal vertex where reverse inversion fails. Consider the root-to-u^* path P.

For every ancestor $a \in P$ at which the path continues into the *right* child, the entire left-child subtree was completed before the algorithm entered the right subtree. Store the root label of that left subtree. Finally store the labels of the two children of u^*. This is essentially Li's description D_h: the proper left children of the path, together with the right child of the failure vertex [12].

Lemma 7.1 (Succinct history)**.** *The canonical reverse-GGM history has a unique description of $O(mh) = O(m^2)$ bits. Given the description and the address of any tree vertex, its history value—an m-bit string or $\perp$—can be computed in polynomial time in m.*

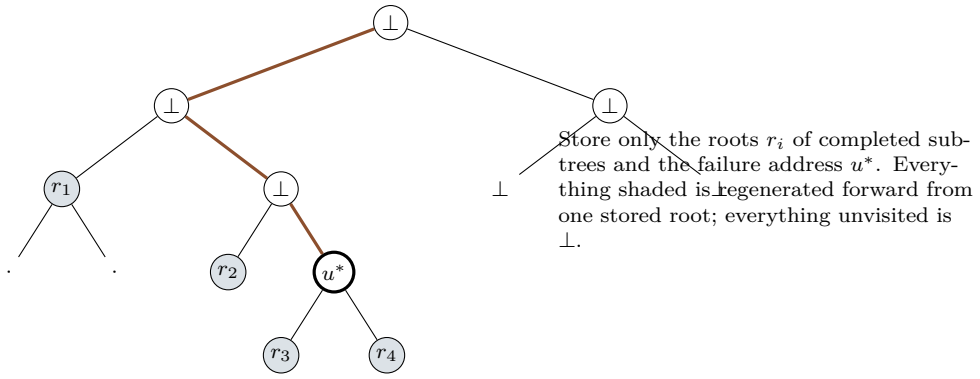
Proof. The path P has at most $h + 1 = O(m)$ vertices, so we store only $O(m)$ roots, each with an m-bit label, plus their addresses.

To recover a requested value, first special-case the leaves: their values are explicitly determined by F_m. For an internal vertex v:

- if v is an ancestor of u^*, it was not successfully assigned and its history value is $\perp$;
- if v lies in one of the completed subtrees whose root value is stored, follow the path from that stored root to v, applying the appropriate half C_L or C_R at each step;
- if the lowest common ancestor of v and u^* sends u^* left and v right, then v's internal node occurs after u^* in postorder and was never assigned, so its value is $\perp$;
- if v is below u^*, one of the two stored children of u^* is an ancestor of v, and again forward GGM evaluation recovers its value.

The computation follows only a root-to-vertex path of length $O(h) = O(m)$, hence takes polynomial time. The description is unique because the reverse algorithm and its lexicographic choices are deterministic. □

The picture to keep in mind is the following. The thick path leads to the failure vertex. The shaded subtrees are completely processed and each needs only its root label stored.



This is the essential Li observation. Chen–Hirahara–Ren had to argue, by an iterative win–win over scales, that some enormous history was compressible. Postorder makes the worst-case history compressible directly.

8 One universal quantifier checks the whole history

A short description is useful only if we can certify that it is the *correct* history. Fortunately, each reverse-GGM step is local.

Lemma 8.1 (Universal verification)**.** *There is a deterministic polynomial-time verifier V such that the genuine history description H_C is the unique string satisfying*

$$\forall w \qquad V(C, H_C, w) = 1,$$

where $|H_C|, |w| = \mathrm{poly}(m)$. From H_C one reads a canonical string $y_C \notin \mathrm{Im}(C)$.

Proof. A universal challenge w specifies which local test to perform, a tree vertex when needed, and an m-bit candidate preimage when needed. Using Theorem 7.1 to recover local history values, check the following.

1. **Leaves.** Every leaf value agrees with the corresponding m-bit block of the explicit string F_m.
2. **Successful reverse steps.** For every internal vertex u occurring before u^* in postorder, if its label is v_u and its children are a, b, then
$$C(v_u) = ab.$$
3. **Lexicographic canonicity.** For every successful vertex u and every $x <_{\mathrm{lex}} v_u$,
$$C(x) \neq ab.$$
4. **Genuine failure.** If the children of u^* are a^*, b^*, then for every $x \in \{0,1\}^m$,
$$C(x) \neq a^*b^*.$$
Thus $y_C := a^*b^* \notin \mathrm{Im}(C)$.
5. **No work after failure.** Every internal vertex at or after u^* in postorder has history value $\perp$.

Each individual test is polynomial-time. One universal string w can encode both the test type and all quantified local choices, so the five families of conditions collapse into one universal quantifier.

The true deterministic reverse computation satisfies all checks. Conversely, the checks force exactly the same leaves, then exactly the lexicographically first preimage at every successful step, then the first genuine failure. Hence they uniquely determine the history and its description. □

The quantifier trade. The literal reverse algorithm makes exponentially many NP-oracle calls. Instead of *running* those calls, we existentially write down the canonical history and use one universal quantifier to certify all negative information at once: "there is no smaller preimage here" and finally "there is no preimage at all here." Postorder is what makes the existential object short enough to write down.

9 Canonical Range Avoidance for stretch one

Combining Theorems 4.1 and 8.1 gives exactly the form needed for circuit lower bounds.

Theorem 9.1 (Canonical $\exists\forall$ Range Avoidance)**.** *There is a polynomial p and a deterministic polynomial-time verifier W such that, for every circuit*

$$G : \{0,1\}^m \to \{0,1\}^{m+1},$$

there is a unique certificate $\pi_G \in \{0,1\}^{\leq p(m+|G|)}$ satisfying

$$\forall w \in \{0,1\}^{\leq p(m+|G|)} \qquad W(G, \pi_G, w) = 1.$$

The certificate determines a canonical string

$$y_G \in \{0,1\}^{m+1} \setminus \mathrm{Im}(G).$$

Proof. Construct the doubling circuit $D : \{0,1\}^m \to \{0,1\}^{2m}$ from Theorem 4.1. The certificate first contains the unique succinct reverse-GGM history for D, universally certified by Theorem 8.1; this yields a canonical $y \notin \mathrm{Im}(D)$. Append the canonical reverse chain from Theorem 4.1, ending at the first target with no preimage under G. Universally verify all successful lexicographically first preimage claims and the final failure claim. Every choice is deterministic, so the combined certificate is unique. □

This theorem is essentially the *single-valued* $F\Sigma_2^P$ portion of Li's argument, specialized and written without functional-class notation. Li's stronger $F\mathsf{S}_2^P$ result adds a selector that can recognize the correct history when confronted with two candidates; that extra symmetry is what places the final language in $\mathsf{S}_2\mathsf{E}$, rather than merely in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$.

10 From canonical Avoid to the circuit lower bound

We can now prove Theorem 1.1 in a few lines.

Proof of Theorem 1.1. For each sufficiently large n, put

$$N = 2^n, \qquad s = \left\lfloor \frac{N}{n} \right\rfloor.$$

By Theorem 3.2, the truth-table generator $\mathsf{TT}_{n,s}$ has fewer than N input bits. Pad it with ignored dummy bits to obtain

$$G_n : \{0,1\}^{N-1} \to \{0,1\}^N.$$

Apply Theorem 9.1 and let $y_n \notin \mathrm{Im}(G_n)$ be its canonical missing output. Define the length-n truth table of L to be y_n.

If an n-input circuit of size at most s computed L at length n, its stack-program description would be an input to G_n whose output is y_n, contradicting $y_n \notin \mathrm{Im}(G_n)$. Hence

$$C_L(n) > s = \left\lfloor \frac{2^n}{n} \right\rfloor.$$

It remains to place L uniformly. On input $x \in \{0,1\}^n$, the machine can construct the relevant truth-table-generator circuitry in $\mathrm{poly}(N) = 2^{O(n)}$ time. It existentially guesses the unique canonical Avoid certificate π_{G_n}, whose length is $\mathrm{poly}(N) = 2^{O(n)}$, and universally quantifies over the challenge w from Theorem 9.1. If the certificate is valid, it reads the bit of y_n indexed by x.

Thus

$$x \in L \iff \exists \pi \, \forall w : \; W(G_n, \pi, w) = 1 \text{ and the } x\text{th output bit encoded by } \pi \text{ is } 1,$$

with all strings and computation bounded by $2^{O(n)}$. Therefore $L \in \Sigma_2\mathsf{E}$.

Because the valid certificate is unique, the complementary condition is identical except that the output bit is required to be 0. Hence $\overline{L} \in \Sigma_2\mathsf{E}$, i.e. $L \in \Pi_2\mathsf{E}$ as well. This proves $L \in \Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$. □

Finally, Frandsen–Miltersen prove

$$\max_f C(f) \le \frac{2^n}{n}\left(1 + 3\frac{\log n}{n} + O\left(\frac{1}{n}\right)\right),$$

so the lower bound in Theorem 1.1 is within a $1 - o(1)$ multiplicative factor of the absolute maximum [8].

What was actually used? No PCP theorem, no hardness-versus-randomness theorem, no pseudodeterminism theorem, and no iterative win–win. The novel ingredients are only:

(i) a GGM tree that forces some local inversion failure;

(ii) lexicographically first preimages, to make the failure canonical;

(iii) postorder, to make the enormous history a union of only $O(m)$ complete subtrees;

(iv) local universal verification of the compressed history.

Everything else is circuit encoding and scaling.

11 What Li adds to reach symmetric exponential time

The proof above already captures the new geometric/combinatorial idea in Li's work, but it stops at a canonical $\exists\forall$ certificate. Li proves a stronger single-valued $F\mathsf{S}_2^P$ algorithm for Avoid [12].

The extra issue is symmetry. A Σ_2-style computation has a privileged existential witness: there is one correct history description that survives all universal challenges. An S_2-style computation must also admit a dual witness that forces the same answer against every existential claim.

Once histories are succinct, Li builds a *selector*: given two candidate history descriptions, one of which is promised to be correct, it finds the correct one in polynomial time. If the two histories differ at some internal vertex, descend toward a differing child; eventually one reaches a leaf, whose true value is fixed by the explicit string F_m, or one finds that two competing preimages of the same target are ordered lexicographically and the smaller one must be correct. The descent takes only the tree height, $O(m)$, because history values can be evaluated from their short descriptions.

This selector upgrades the one-sided canonicality proved above to symmetric canonicality and yields Li's stronger theorem: a language in $\mathsf{S}_2\mathsf{E}$ has near-maximum $2^n/n$ circuit complexity almost everywhere. Li also obtains the same almost-everywhere lower bound in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$ and in zero-error exponential time with an NP oracle. The postorder simplification is what removes the one bit of advice in the earlier Chen–Hirahara–Ren theorem [11, 12].

12 The Korten–Pitassi perspective: canonicalization is an ordering problem

Korten and Pitassi ask what Li's algorithm is *really doing*. Their answer is the Linear Ordering Principle (LOP) [13].

Definition 12.1 (Linear Ordering Principle)**.** An instance is a circuit $\prec\colon \{0,1\}^r \times \{0,1\}^r \to \{0,1\}$ purporting to describe a total order on $\{0,1\}^r$. One must output either

- a local witness that $\prec$ is not a total order (failure of irreflexivity, comparability, or transitivity), or
- if it really is a total order, its unique minimum element.

LOP has "essentially unique" solutions: malformed orders have easily verifiable local violations, while a genuine total order has one distinguished minimum. Korten–Pitassi prove that Weak Range Avoidance reduces to LOP. Their conceptual decomposition of Li's argument is particularly useful:

1. **Prepare a special family of candidate solutions.** The tree-like GGM iteration restricts attention to missing outputs arising as canonical local failures of a structured inversion process.
2. **Choose one canonically.** Define a succinct total order on these candidates and take its minimum.

This explains why a search problem with exponentially many apparent solutions can nevertheless behave as if it had a distinguished low-complexity solution. Korten–Pitassi define a corresponding class $L_2^P \subseteq \mathsf{S}_2^P$, show $MA \subseteq L_2^P$, and show that its exponential analogue L_2^E already has a $2^n/n$ circuit lower bound [13].

For teaching the circuit lower bound, however, the direct postorder proof above is shorter: LOP is best viewed as the *structural explanation after the proof*, not as a prerequisite.

Korten–Pitassi also show that the picture is special to Weak Avoid: an analogous Strong Avoid problem resists these upper-bound mechanisms in a black-box decision-tree sense, via a new depth-three AC^0 lower bound for a total search problem. This is evidence that the Li phenomenon is not a generic consequence of every pigeonhole search problem having many solutions.

13 Where the iterative win–win went

The Chen–Hirahara–Ren proof predates the postorder observation. Their basic inversion history is locally checkable but potentially exponentially long. They use an iterative win–win paradigm to range over scales: either a certain construction already yields the desired hard object, or failure at one scale gives enough structure to compress the next computation. After iterating, some scale has a short enough canonical history. Because the good scale is not known uniformly, the original theorem uses one bit of advice and gives the $\mathsf{S}_2\mathsf{E}/_1$ lower bound infinitely often [11].

Li's observation is that the apparent need to hunt for a compressible scale was an artifact of traversal order. In postorder, the largest, worst-case GGM tree already has a succinct history. Thus the win–win is bypassed rather than improved.

A useful methodological lesson. The breakthrough is not a stronger circuit-counting argument. It is an *algorithmic representation theorem about a failed inversion process.* By changing the order in which a deterministic computation explores an enormous tree, one changes the descriptive complexity of its partial execution from exponential to polynomial. That compressed canonical execution is then a circuit lower bound.

14 The current frontier

The 2023–24 work moved near-maximum circuit complexity dramatically downward, from the classical $\Delta_3\mathsf{E}$ construction to symmetric/second-level exponential classes. Several subsequent works have treated Range Avoidance as a central object in its own right, connecting it to explicit constructions, proof complexity, bounded arithmetic, cryptography, and restricted-circuit algorithms.

A particularly recent endpoint is due to Ren and Williams: they prove a near-maximum $2^n/n$ lower bound for $\mathsf{E}^{\mathrm{prMA}}/_1$, exponential time with a promise-MA oracle and one advice bit [14]. Unlike the stripped-down proof in this note, their argument again needs an iterative win–win and also uses PCP-encoded computation histories. This is a stronger endpoint but not a good candidate for a single self-contained lecture.

For a first encounter, the postorder proof is unusually well positioned: it exposes the modern Range-Avoidance viewpoint and proves a genuinely near-maximal lower bound in a small class without importing a major technical theorem.

15 An 80-minute lecture plan

Assume students already know Σ_2-style quantifiers, exponential-time notation, and basic circuits. The following timing is realistic.

Minutes	Content
0–8	Shannon scale, truth-table generator, and why a missing output is a hard truth table.
8–17	Range Avoidance; statement of the target $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$ lower bound.
17–25	Stretch-one to doubling gadget (Theorem 4.1).
25–35	GGM tree and the explicit impossible leaf string F_m.
35–47	Reverse inversion in postorder; why failure produces a missing output.
47–61	Li's key lemma: the failed history is only $O(m)$ complete subtrees; draw the path picture and prove succinct evaluability.
61–70	Universal verification, lexicographically first preimages, uniqueness.
70–77	Apply to the truth-table generator; scale $m = 2^n$ back to $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$.
77–80	Historical coda: CHR iterative win–win, Li's $\mathsf{S}_2\mathsf{E}$ selector, Korten–Pitassi LOP.

If the exact constant $2^n/n$ is not important, the stack-program encoding in Section 3 can be assigned beforehand and the lecture can begin directly with Range Avoidance. Conversely, if one wants the precise Shannon-scale comparison, the Frandsen–Miltersen encoding is a particularly nice short preliminary theorem.

16 Summary: the proof in seven sentences

1. Encode every size-$2^n/n$ circuit in fewer than 2^n bits, and let G_n map descriptions to truth tables.
2. A missing output of G_n is a near-maximally hard Boolean function.
3. Reduce a stretch-one circuit $G : \{0,1\}^m \to \{0,1\}^{m+1}$ to a doubling circuit $C : \{0,1\}^m \to \{0,1\}^{2m}$.
4. Apply C down a height-$2m+1$ binary tree; the leaf string enumerating every $2m$-bit string cannot be generated by any root seed.
5. Invert the tree in postorder using lexicographically first preimages, and stop at the first local pair with no preimage.
6. Postorder makes the enormous partial history a union of $O(m)$ fully generated subtrees, so one can existentially describe it in $\mathrm{poly}(m)$ bits and universally verify every local step.
7. At truth-table length $m \asymp 2^n$, this $\exists\forall$ certificate costs only $2^{O(n)}$, yielding a canonical near-maximally hard language in $\Sigma_2\mathsf{E} \cap \Pi_2\mathsf{E}$.

References

[1] C. E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28:59–98, 1949.

[2] O. B. Lupanov. The synthesis of contact circuits. *Doklady Akademii Nauk SSSR*, 119:23–26, 1958.

[3] R. Kannan. Circuit-size lower bounds and non-reducibility to sparse sets. *Information and Control*, 55:40–56, 1982.

[4] N. Nisan and A. Wigderson. Hardness vs. randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994.

[5] R. Impagliazzo and A. Wigderson. $P = BPP$ if E requires exponential circuits: derandomizing the XOR lemma. In *STOC*, pages 220–229, 1997.

[6] P. B. Miltersen, N. V. Vinodchandran, and O. Watanabe. Super-polynomial versus half-exponential circuit size in the exponential hierarchy. In *COCOON*, LNCS 1627, pages 210–220, 1999.

[7] E. Jeřábek. Dual weak pigeonhole principle, Boolean complexity, and derandomization. *Annals of Pure and Applied Logic*, 129:1–37, 2004.

[8] G. S. Frandsen and P. B. Miltersen. Reviewing bounds on the circuit size of the hardest functions. *Information Processing Letters*, 95(2):354–357, 2005.

[9] O. Goldreich, S. Goldwasser, and S. Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986.

[10] O. Korten. The hardest explicit construction. In *FOCS 2021*, pages 433–444. IEEE, 2022.

[11] L. Chen, S. Hirahara, and H. Ren. Symmetric exponential time requires near-maximum circuit size. In *STOC 2024*. See also ECCC TR23-144.

[12] Z. Li. Symmetric exponential time requires near-maximum circuit size: simplified, truly uniform. In *STOC 2024*. See also ECCC TR23-156, Revision 1.

[13] O. Korten and T. Pitassi. Strong vs. Weak Range Avoidance and the Linear Ordering Principle. ECCC TR24-076, 2024.

[14] H. Ren and R. R. Williams. Near-maximum circuit lower bounds for exponential time with Merlin-Arthur queries. ECCC TR26-118 / arXiv:2607.09963, 2026.